

Homework6 CS5800

Rayan Hassan

February 2025

1 Exercise

(a)

Assuming that x is not the largest value in the BST (meaning it does have a right subtree), we can first look right of x , in other words we look to the root of its rightmost subtree. And then we keep looking to the left (leftmost nodes), until we reach a leaf. This leaf will be for sure smaller than all of its parents but larger than x (because we started by drifting to the right of x). So the algorithm is: start by looking to the right of x . If this element has no left subtree then this is the result we are looking for. Else, look to the left subtree of this element and keep going the same way (meaning looking to the left) until you reach a leaf. This is like the "find successor and replace" method for deleting in a BST. The worst case scenario here is if x is at the root and the element to its right has only a left child node, which itself only has 1 child node, which itself only has 1 child node, etc. Now the time complexity of going to the right of x the first time is just $O(1)$, and then the time to search for the leftmost element until we hit a leaf is $O(\text{height of the subtree that we are looking for})$ which in this case is $O(h-1)$ or just $O(h)$. h here is the height of this subtree (which really just looks like a linked list in the worst case), so $h=n-1$ (in the worst case!). This means that the time complexity of this algorithm is $O(n-2)$, so really it is $O(n)$. But this is only if the subtree is fully skewed (worst case). If it was balanced for example, the height would be $O(\log n)$. So in conclusion, in general, the time complexity of this algorithm is $O(h)$.

(b)

In this case, we start by finding the first successor of x . So if x has a right child, go to this child and find the leftmost node in that subtree, which gives you the smallest value larger than x . But to find this first successor, it takes $O(h)$, because in the worst case this can be the height of the whole tree if it is skewed. Now once the first successor is found, there is no need to traverse the tree again. So what we do is we simply get the next $k-1$ successors. Which means that in total this algorithm would take $O(h+k)$.